TECHNICAL REPORT

TPS-Benchmarking Full Node

Phase 3 — The Optimization PR

Measuring what the Rust-verification & consensus optimizations buy a Hathor full node, transparent and shielded

July 3, 2026

Luis Felipe Silva Rezende Soares

Abstract

This third phase measures the effect of a large upstream performance change — **PR #1729 ("rust script executor")** — on the single-thread transaction-processing rate of a Hathor full node, and does so for **both transparent and shielded (confidential) transactions**. We brought the PR's optimizations into the benchmark engine **behind runtime flags** (`--opt` / `--no-opt` / per-section), so every optimized path has a baseline twin and the two can be measured back-to-back on the same node. Using the same in-process, white-box, single-thread methodology as Phase 1 (a real `HathorManager`, real RocksDB, real verification and consensus, transactions driven through six timed stages S1–S6), we ran eleven experiments across transaction shape and flag configuration.

On the reference machine (Intel i5-11300H, single thread) the optimized node processes a baseline 1-input / 2-output transparent transaction at **≈ 640–790 tx/s**, versus **≈ 200 tx/s** with every optimization disabled — a **≈ 3× speed-up** (larger, up to ~4.3×, at smaller batch sizes on a less-loaded machine). Turning the sections off one at a time shows the win is **dominated by S5 (consensus + storage)**, followed by **S3S4 (Rust script verification)** and **S6 (dropping the redundant second full validation)** — independently reproducing the bottleneck ranking Phase 1 predicted. Shielded transactions are a different regime entirely: **≈ 9× slower** than transparent (≈ 71 tx/s at 1i2o), with **≈ 90 % of the cost in a single stage — range-proof verification (S3S4)** — and the expensive axis is **outputs, not inputs** (each shielded output is one more range proof), the mirror image of the transparent case. Critically, **the PR's verification parallelization does not apply to shielded transactions** — they fall back to a serial path — so shielded throughput reflects a **serial, per-proof verification ceiling** and benefits little from `--opt`. Three upstream defects in the shielded-outputs stack were found and fixed to make these measurements possible. As in Phase 1, every figure is a **single-thread, single-machine** number and must be scaled to other hardware; the WSL host makes absolute numbers volatile, so the ratios and per-stage attributions — not the absolutes — are the result.

1. Introduction & Motivation

The pipeline, recapped. Every transaction that reaches the node flows through a fixed sequence of stages; the whole study attributes each measured microsecond to one of them. We keep the S1–S6 labels from Phase 1 and give each a short tag used throughout the figures:

Stage	Tag	What it does
S1	`ser/de`	Deserialize: raw bytes → vertex object
S2	`gate`	Pre-checks / gatekeeping: already-known, double-spend, spends-voided, reward-lock
S3+S4	`verify`	Full verification: PoW check, input signatures / scripts, balance (and, for shielded, range + surjection proofs)
S5	`cons.`	Save & consensus: RocksDB write, mark inputs spent, voided-status, **mempool-tips index**, indexes
S6	`post-cons.`	Post-consensus: finalize indexes/events — and, in the baseline, a **second** full validation

This report first explains **what the PR changes** (§2), then the **method and flag-gating** (§3), the **limits** of what the numbers mean (§4), the **eleven experiments** with their conclusions (§5), the **shielded bugs** that had to be fixed first (§6), and closes with **what it all implies** (§7).

2. The optimizations under test (PR #1729)

The PR rewrites the hot path in **Rust** and then, once verification stops being the bottleneck, attacks the consensus/storage layer. Its own roadmap reports **~980 tx/s (pure Python) → ~3,649 tx/s (≈ 3.7×)**. The changes map cleanly onto our stages:

Stage	What the PR changes	Nature
S1 `ser/de`	A **Rust vertex parser** (native hashing, reused origin bytes). Declines header-carrying vertices (nano / fee / **shielded**) and falls back to Python.	One gated branch; tiny (~0.8 µs/tx).
S2 `gate`	Lock-free **LRU fast path** for `get_transaction`, scope fusion, a miss-probe skip; stateless checks in Rust.	Localized read fast-paths.
S3S4 `verify`	A **Rust script interpreter** run as a **single GIL-released, fused batch call**, plus a **parallel executor** (rayon / process / thread pools) that fans input-script checks across cores.	**Consensus-critical**; the headline rewrite.
S5 `cons.`	**Rust-owned RocksDB**, **binary metadata** serialization (replacing JSON), and an **incremental mempool-tips** update (spender-first, `is_new` hint) plus save-dedup and `WriteBatch` flushing.	The bulk of the later gains.
S6 `post-cons.`	**Drops the redundant second** `validate_full`, writes info-index counters **only on change**, and batches reactor yields.	Removes duplicated work.

Two facts about the PR shape everything below.

(a) It moved the bottleneck. After the Rust rewrite, all verification is **~4.8 µs/tx ≈ 2 %** of the per-tx budget, so the largest remaining gains are in **S5 (consensus + storage)** — exactly the layer Phase 1 independently fingered (the O(tip-count) `mempool_tips.update`). The PR also removes the **second** `validate_full`, the top single-thread lever Phase 1 identified.

(b) It predates shielded transactions. PR #1729 was written against **pre-shielded master**. Its verification fast paths therefore **do not understand shielded vertices**: the Rust parser declines them (S1 → Python), and — most importantly — the **parallel script executor falls back to a serial path for any transaction that spends or creates a shielded output** (`transaction_verifier.py`: "Shielded txs always take the serial path"). The heavy shielded cryptography (Borromean range proofs, surjection proofs, in the native `secp256k1-zkp` crate) is therefore **verified serially, one proof at a time, on a single thread** — neither the baseline nor the PR parallelizes it. This is the single most important caveat for reading the shielded results, and we restate it wherever it applies.

3. Methodology

3.1 Machine specifications

All measurements were taken on a single reference machine (the same as Phase 1):

Component	Specification
CPU	Intel **Core i5-11300H** (Tiger Lake, 11th gen), 4 cores / 8 threads, 3.10 GHz base / 4.40 GHz boost
Cores used	**One core** for the measured path; the Rust script pool may use up to **4 worker threads** for transparent input verification (see §4)
RAM	**≈ 12 GB**
OS	**Windows 11** host, **WSL 2** (Ubuntu); Python 3.11
Storage	RocksDB on a temporary directory (NVMe-backed), fresh per run

> ⚠ **Every throughput figure is specific to this hardware and MUST be scaled to the reader's own machine.** Processing is single-thread CPU-bound, so the rate scales with single-thread CPU performance, not core count: `TPS_target ≈ TPS_here × (single_thread_score_target / single_thread_score_i5-11300H)`.

3.2 How the optimizations are measured (flag gating)

Rather than run two code-bases, we carry **both the baseline and the optimized implementation in the tree** and select the live one at runtime. `--opt` turns all five sections on (the default); `--no-opt` turns them all off (≈ the pre-merge node); `--no-opt sX` runs **section X in baseline while the rest stay optimized**, which **isolates that section's marginal contribution**. The harness exports the choice to the node before each build, and the sections gate at their exact call sites, keeping both paths intact.

3.3 The workloads

Two workloads are used, both 1-tip (tips ≈ 1) so per-tx cost is steady:

- **`1-tip-transparent`** — the Phase-1 construction: coinbase blocks → chained `fund` transactions minting pinned UTXOs → payload transactions each spending their own disjoint slice, parent-chained. Every Hathor transaction confirms two "parents"; if a synthetic workload let every transaction parent genesis, then no transaction is ever another's parent, so **all N transactions are tips**, and because consensus re-scans all current tips on every transaction (`mempool_tips.update` is O(tips)) the per-transaction cost in S5 grows with the batch and the run costs **O(N²)**. Chaining the transactions in the parent DAG — `tx_k` names `tx_{k-1}` as a parent — keeps **only the latest as a tip**: the tip set stays ≈ 1, the tip scan is O(1), and S5 is flat in N, which lets us read a steady per-transaction cost instead of a batch artifact. It is deliberately simpler than mainnet's 2–3-tip mesh (see §4).
- **`capless-full-shielded`** — a fully-confidential (shielded-input **and** shielded-output) workload. Each measured transaction spends pre-minted shielded UTXOs and emits shielded outputs. Because shielded inputs require thousands of pre-minted shielded UTXOs, the sources are funded **explicitly, in chunks**, lifting a 255-output cap that

the naïve approach hits — so shielded input/output counts scale to the full batch. A **build-time range-proof cache** speeds construction only; it never touches verification, which re-verifies every proof (audited: no verify-side dedup).

3.4 Data collection and treatment

For every transaction we record **per-stage wall and CPU time**; a background sampler captures RSS, disk I/O, and file-descriptor counts. Each experiment cell (a workload × shape × flag combination) is run at **N = 2000 measured transactions** (P8 grid and P11 transition at N = 1000), preceded by a **200-transaction warm-up** that is driven but discarded (cold RocksDB cache and interpreter make the opening transactions unrepresentative). Each cell is repeated **k = 3 times** on independent builds, and we report the **median**. The per-tx-index band figures shade the min/avg/max over the three repetitions; because the WSL host and RocksDB compaction inject **rare latency explosions** (a single transaction 20–50× slower than its neighbours), the band plots are **σ-clipped at 3.5σ** — points beyond mean + 3.5σ of their own series are hidden so the real signal is legible, and the count of hidden points is annotated on each figure. Headline throughput is `N / Σ(per-tx total wall)`.

4. Scope & limitations

The figures are honest but bounded. Read them with these caveats, which recur in the sections they affect:

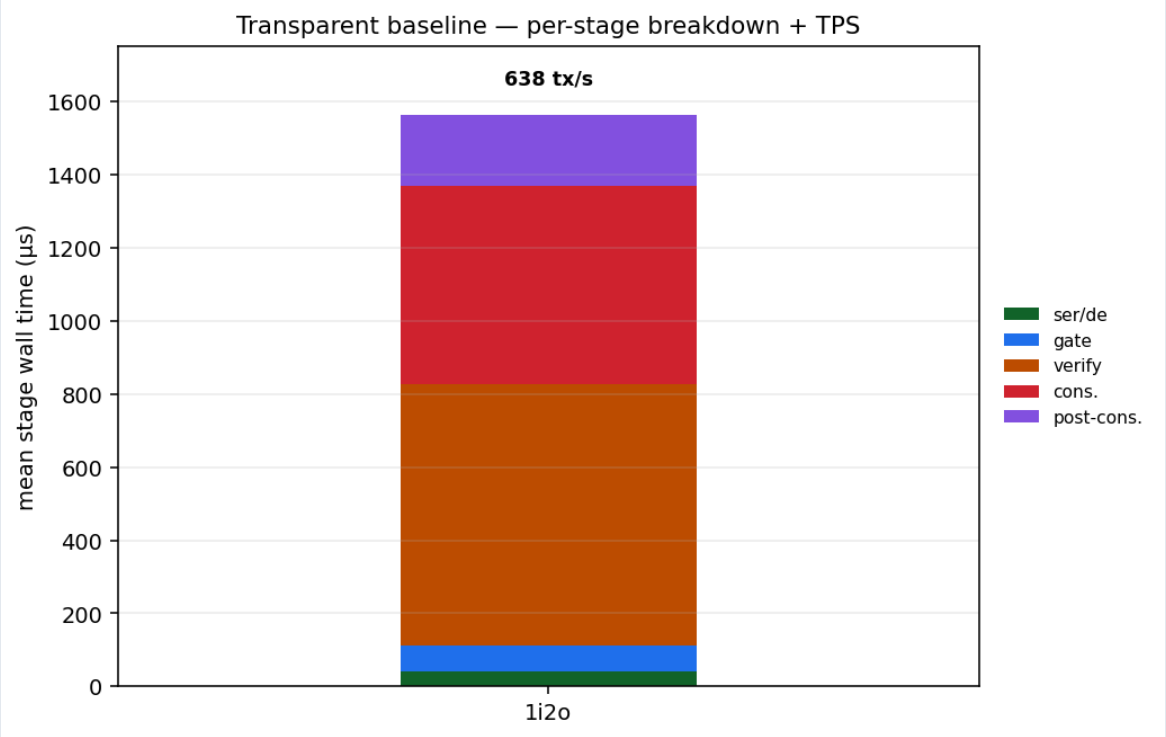
- **Single machine — must be scaled.** Every number is specific to the i5-11300H single-thread performance (§3.1). Treat the absolutes as points on a curve, not universal constants.
- **Time volatility (WSL).** The host is **WSL 2 under load**; run-to-run variance is large (the transparent 1i2o baseline alone ranges ≈ 620–790 tx/s across experiments) and rare per-transaction write-stall / GC spikes require the σ-clip to interpret. **The ratios and the per-stage attributions are the result; absolute TPS is indicative only.**
- **Shielded verification is not parallelized (stress).** The PR's parallel/Rust script executor applies only to **transparent** input verification; **any shielded transaction falls back to a serial path**, and the native range-proof / surjection verification is single-threaded per proof regardless. So the shielded numbers below are a **serial verification ceiling** — they neither reflect nor benefit from the parallelization that helps transparent transactions, and `--opt` moves them far less than it moves transparent throughput. Every shielded experiment (§5.3–§5.6) is bounded by this.
- **1-tip DAG, not a mainnet mesh.** Our workload chains transactions into a ≈ 1-tip DAG; live traffic forms a 2–3-tip mesh. This removes the $O(N^2)$ artifact and exposes steady cost, but does not exercise consensus tip-management exactly as mainnet would.
- **Fresh, small database.** We run against a fresh temporary RocksDB (RSS ≈ 0.15–2.5 GB depending on shielded proof volume), so the numbers do **not** reveal the RAM ceiling or cache-miss penalties a mainnet-sized UTXO set would impose on verification reads.
- **P11 (transition) is partial.** The transition experiment completed 2 of 6 configurations; the heavier shielded shapes hit a source-funding cap in the multi-batch builder (a build-side limit, not a node limit) and are pending a fix. Its results are reported as preliminary.

5. Results

Each subsection is one experiment: its figure(s), a summarized table, and the conclusion — what we observed and why, in terms of the pipeline stages. Unless noted, all cells are N = 2000, k = 3, median, optimized (`--opt`), 64-bit range proofs.

5.1 Transparent baseline

The optimized node processes a 1-input / 2-output transparent transaction at **≈ 640 tx/s** (this run; ≈ 640–790 across the campaign). The per-stage split shows the cost is **not** in any one place — it is shared between **verification (S3S4)** and **save + consensus (S5)**, with post-consensus (S6) third:

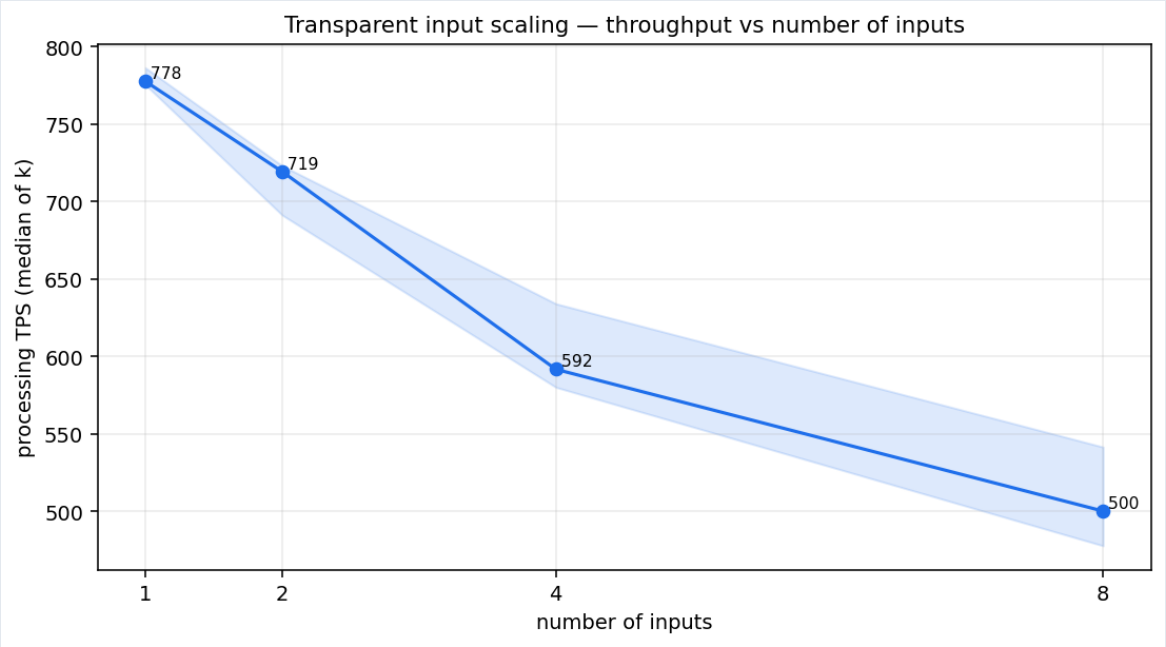


Stage	ser/de	gate	verify	cons.	post-cons.	Total
Mean wall (µs)	42	68	717	543	193	≈ 1 570

Conclusion. With the optimizations on, verification and consensus are of comparable weight and no single stage dominates — the double-verification and O(tips) consensus costs that dominated the un-optimized Phase-1 baseline have been cut down (see §5.9–§5.10). This is the reference point for everything else.

5.2 Transparent input scaling

Sweeping inputs 1 → 8 (outputs fixed at 2), throughput falls from **778 → 500 tx/s** (≈ −36 %):

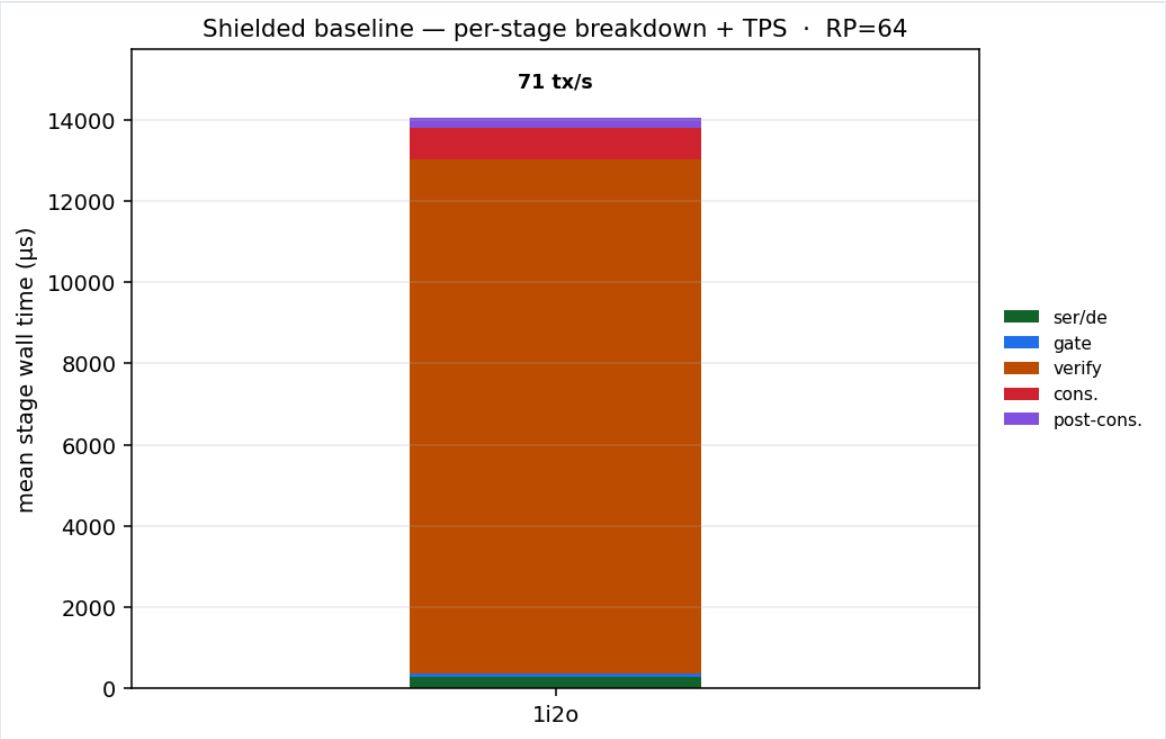


Inputs	1	2	4	8
TPS	778	719	592	500
`verify` S3S4 (µs)	580	695	859	1 188

Conclusion: The Rust scripts and multi-processing reduced the previously hefty cost that an increase of inputs brought to the TPS rate of the full-node. Input scaling reduces the transactions-per-second count, yet no longer plummets it.

5.3 Shielded baseline

A fully-shielded 1-input / 2-output transaction processes at **≈ 71 tx/s — about 9× slower than its transparent twin.** The per-stage breakdown is stark:

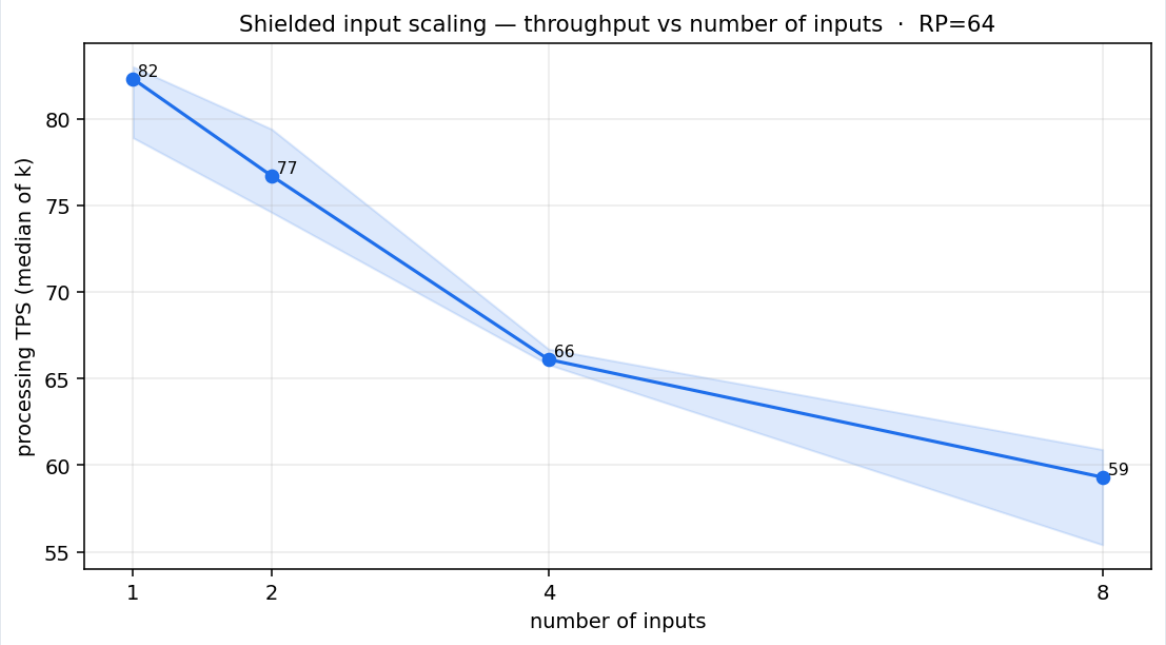


Stage	ser/de	gate	verify	cons.	post-cons.	Total
Mean wall (µs)	278	98	**12 647**	777	246	≈ 14 050

Conclusion — privacy is a verification tax, paid serially. ≈ 90 % of the entire per-transaction budget is a **single stage,** `verify` **(S3S4)** — the Borromean **range proofs** and **surjection proof** that hide the amounts and asset. Consensus and storage barely move relative to transparent; the shielded penalty is all cryptography. And per §4, **this verification is serial** — the PR's parallelization does not reach it — so this ≈ 12.6 ms is a single-thread, per-proof figure with no cross-core relief.

5.4 Shielded input scaling

Sweeping shielded inputs 1 → 8 (outputs fixed at 2), throughput falls only **82 → 59 tx/s (≈ −28 %)** — flatter than the transparent input sweep:

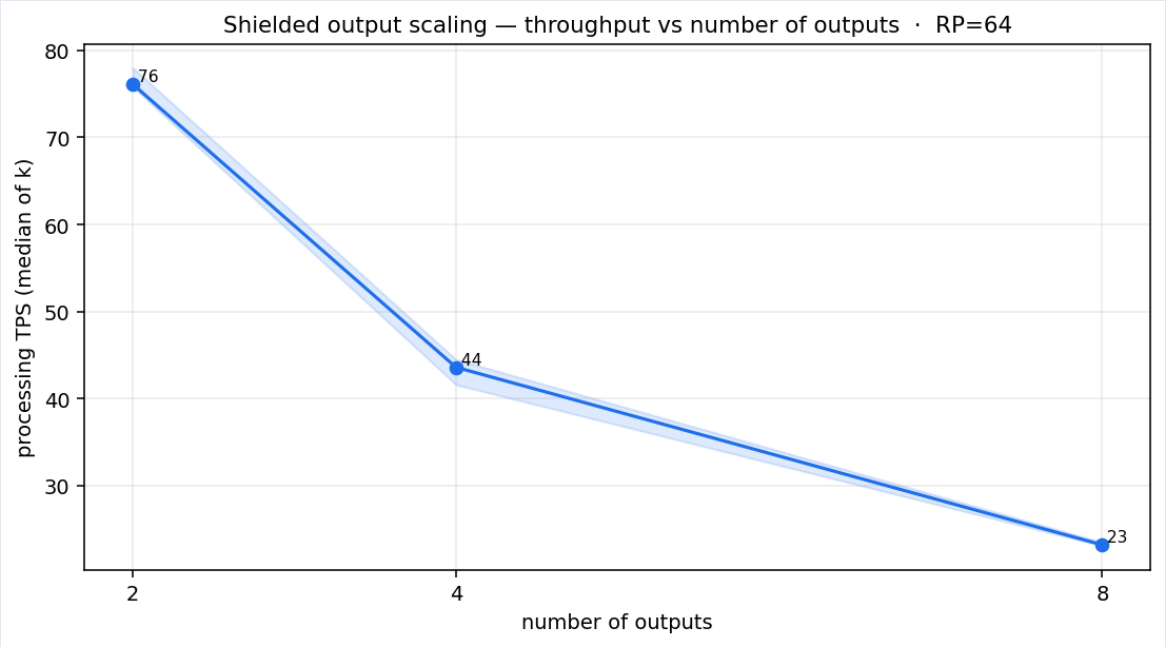


Shielded inputs	1	2	4	8
TPS	82	77	66	59
`verify` S3S4 (µs)	11 083	11 942	13 749	15 586

Conclusion — inputs matter less when you're already paying for proofs. Adding a shielded input grows the **surjection domain** (the proof that each output's asset came from the input set) and the balance fold, but it does **not** add a range proof. Against the ≈ 11 ms fixed range-proof floor of the two outputs, that increment is modest — hence the shallow slope. Inputs are the expensive axis for transparent transactions; for shielded ones they are secondary.

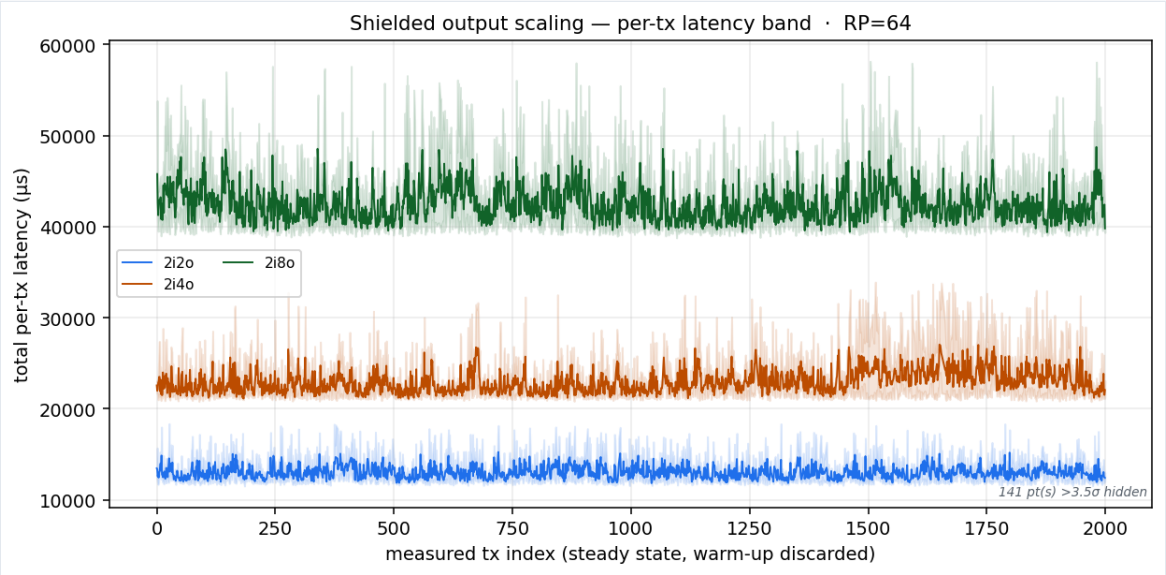
5.5 Shielded output scaling

The mirror finding. Sweeping shielded outputs 2 → 8 (inputs fixed at 2), throughput **collapses 76 → 44 → 23 tx/s**, and `verify` scales almost linearly with output count:



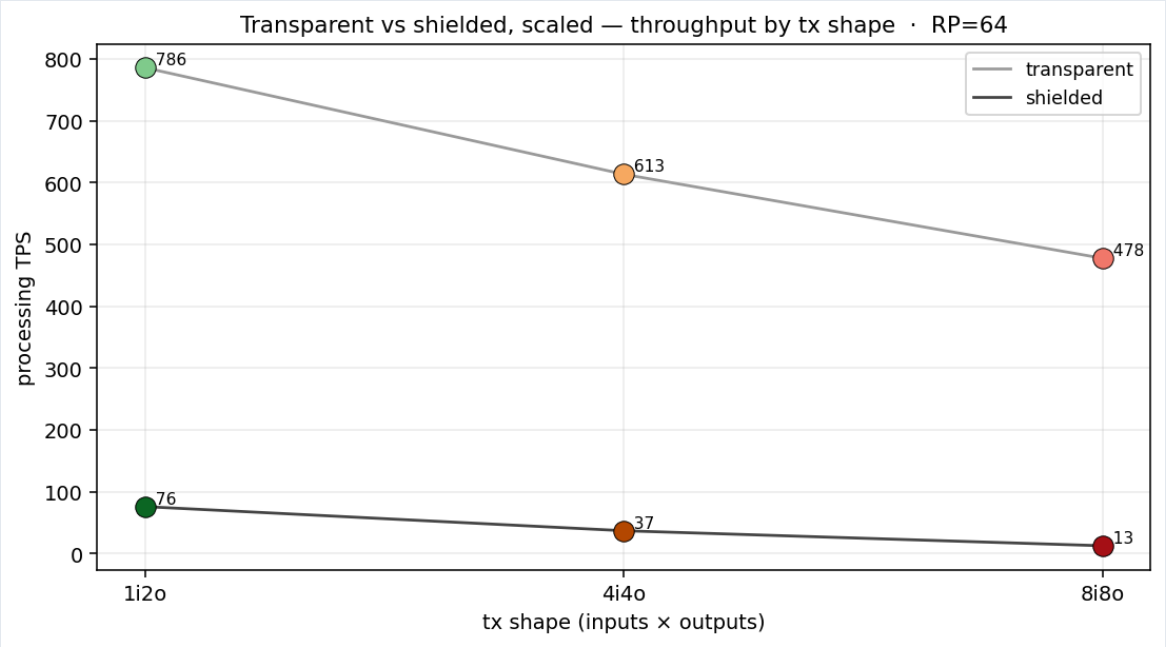
Shielded outputs	2	4	8
TPS	76	44	23
`verify` S3S4 (µs)	12 039	21 281	41 333

Conclusion — outputs are the expensive axis for shielded. Each shielded output is one more Borromean range proof to verify, and range-proof verification dominates the budget, so `verify` grows ≈ +10 ms per doubling and TPS falls roughly as 1/outputs. This is the exact inverse of the transparent case (§5.2, where inputs/signatures dominated). The per-tx-index band below shows the three output counts as cleanly separated latency regimes (with the σ-clip removing the compaction spikes that would otherwise flatten them):



5.6 Transparent vs shielded, at scale

Overlaying the two workloads across shapes makes the gap — and its growth — visible:

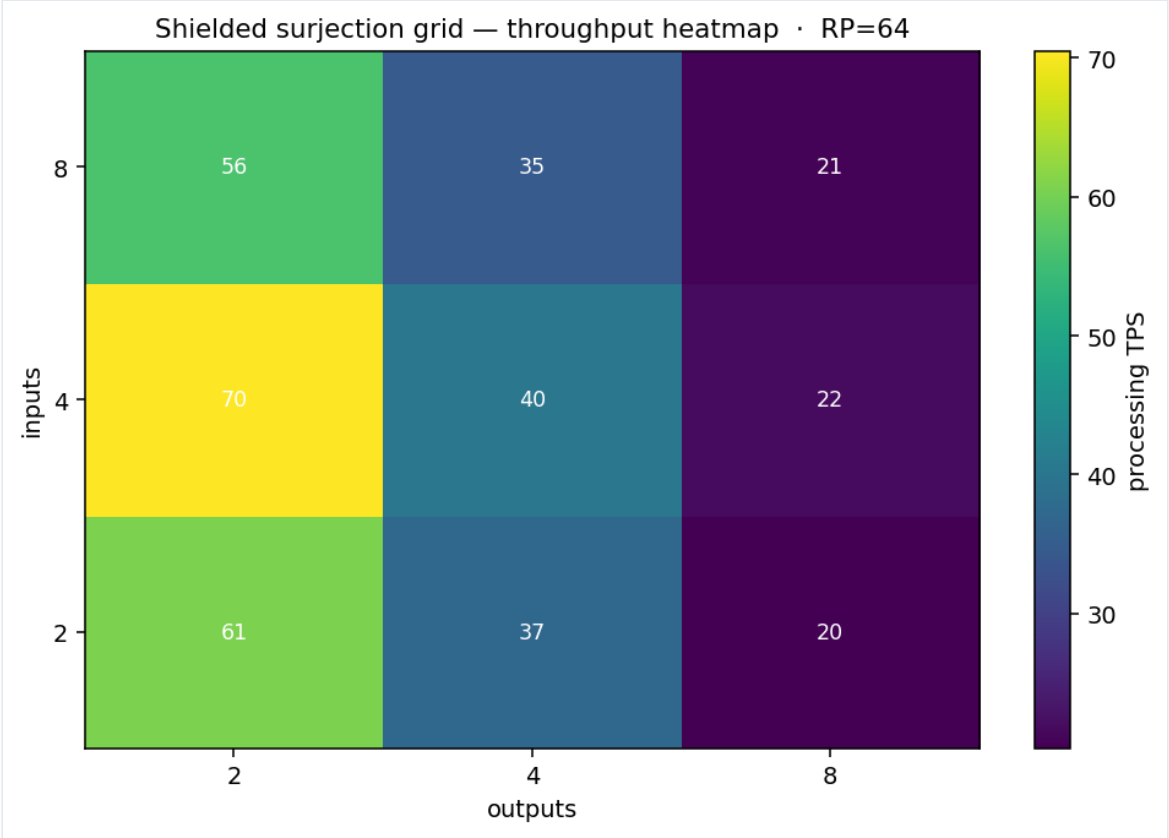


Shape	Transparent TPS	Shielded TPS	Gap
1i2o	786	76	10×
4i4o	613	37	17×
8i8o	477	**13**	**38×**

Conclusion: the gap widens with the increase of elements (inputs/outputs), and this is mostly due to the single-process verification of shielded transactions, since range proofs were not touched by the PR.

5.7 The surjection grid — inputs vs outputs

Crossing inputs {2,4,8} with outputs {2,4,8} isolates which axis governs shielded cost:

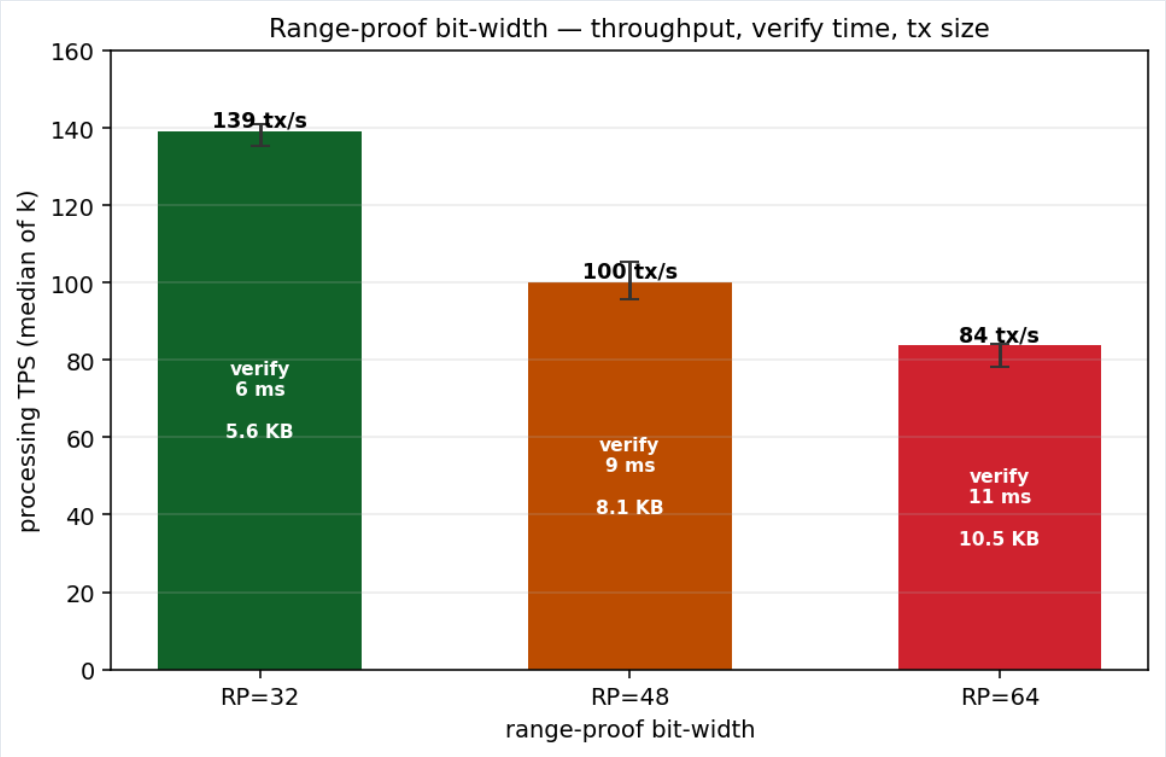


TPS	o=2	o=4	o=8
i=2	61	37	20
i=4	71	40	22
i=8	56	35	21

Conclusion — throughput tracks outputs, almost independently of inputs, since non-parallelized output verification dominates in cost. Reading down a column (fixed outputs) the numbers barely move; reading across a row (growing outputs) they halve each step. This is the clean two-dimensional confirmation of §5.4–§5.5: **shielded cost is set by the number of range proofs (outputs)**, and input count — the surjection-domain size — is a second-order effect. (The mild non-monotonicity across inputs is WSL variance, §4.)

5.8 Range-proof bit-width — the cost / size dial

Every result above uses **64-bit** range proofs — the size of the value range a shielded output can hide. That width is a tunable: a narrower range needs a smaller, cheaper proof. Holding the shielded 1i2o shape fixed and sweeping RP = 32 / 48 / 64:

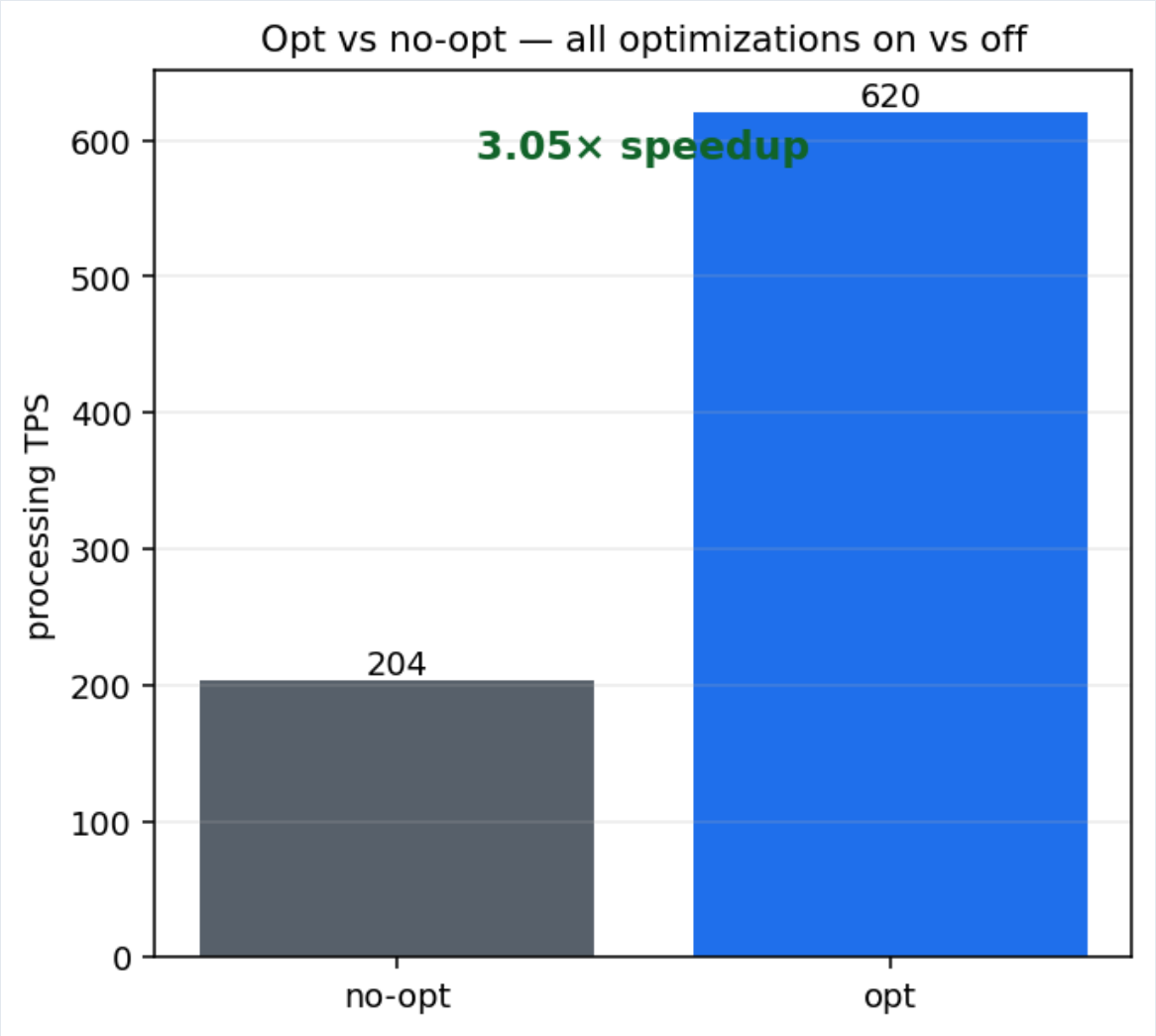


Range-proof bits	32	48	64
TPS	139	100	84
`verify` S3S4	6.2 ms	8.9 ms	10.9 ms
Serialized tx size	5.6 KB	8.1 KB	10.5 KB

Conclusion — bit-width is a clean, near-linear dial on both cost and size. A Borromean range proof commits work per proven bit, so both its **verification time** and the **transaction's serialized size** (which the range proofs dominate) grow roughly linearly with the width — verify rises ≈ +2.4 ms and the tx grows ≈ +2.5 KB per 16 bits, and throughput nearly **doubles** going from 64-bit to 32-bit (84 → 139 tx/s). Because this multiplies the **per-output** proof cost — the axis §5.5 showed to dominate shielded work — it is the most direct lever an application has on shielded throughput short of parallelizing verification (§4, §7): an output that only ever holds a small value can prove a 32-bit range and verify ≈ 40 % faster. (64-bit remains the correct default ceiling for a general amount; narrowing it is a per-application trade-off, not a free win.)

5.9 Optimization payoff — on vs off

The headline of the whole phase. Same transparent 1i2o workload, everything on vs everything off:

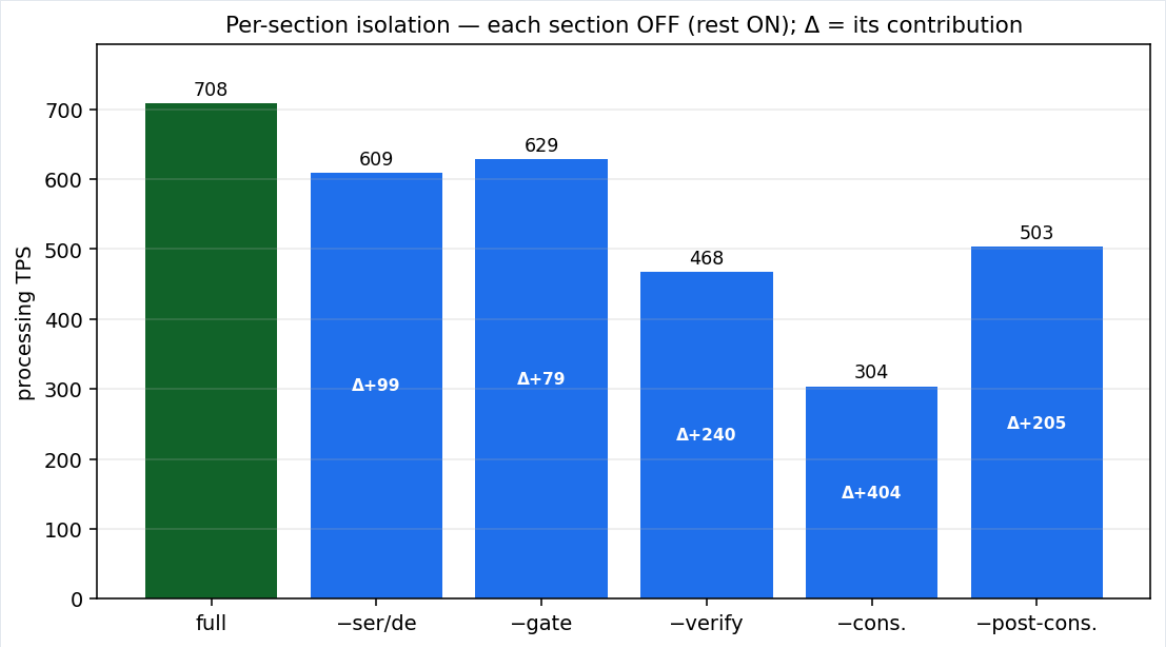


Config	TPS	verify	cons.	post-cons.
`--no-opt` (all off)	203	1 187	**2 177**	**1 331**
`--opt` (all on)	620	735	596	173

Conclusion — ≈ 3× at N = 2000, and it comes from consensus, not verification. Disabling the optimizations triples the per-transaction cost, and the extra cost is overwhelmingly in `cons.` **(S5)** and `post-cons.` **(S6)**: the un-optimized node pays an O(tips) consensus update and re-runs the entire verification a second time in post-consensus. (The ratio is ≈ 3× here and reached ≈ 4.3× on a smaller, less-loaded run — the volatility caveat of §4; the ratio is the result.)

5.10 Per-section contribution

Turning the sections off one at a time (rest optimized) ranks their marginal value:

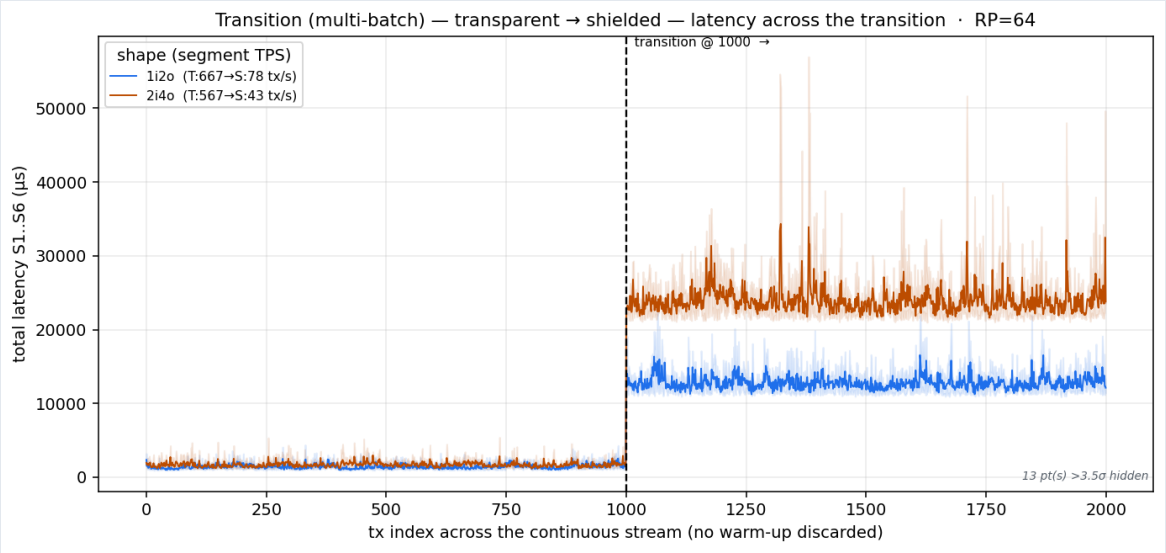


Section off	TPS	Δ from full (708)	Contribution
`-cons.` (S5)	304	**−404**	**biggest**
`-verify` (S3S4)	468	−240	large
`-post-cons.` (S6)	503	−205	large
`-ser/de` (S1)	609	−99	small
`-gate` (S2)	629	−79	small

Conclusion — S5 is the dominant lever, reproducing Phase 1. The consensus + storage rewrite (incremental mempool-tips, binary metadata, Rust RocksDB, save-dedup) is worth the most by a clear margin, with Rust-script verification and the dropped second validation next, and parse/gatekeeping minor. This is the same ranking Phase 1 predicted from first principles — the O(tip-count) consensus update was always the bottleneck. (The Δ's overlap and do not sum to the total: dropping `post-cons.`'s second validation also avoids re-running `verify`, so those two share credit.)

5.11 Workload transition (preliminary)

Driving a **transparent → shielded** stream as one continuous run shows the composition shift in real time: throughput steps down by an order of magnitude the instant the shielded segment begins.



Configuration	transparent segment	shielded segment
T→S, 1i2o	667 tx/s	78 tx/s
T→S, 2i4o	567 tx/s	43 tx/s

Conclusion. The node has no "warm-up" advantage carrying between regimes — the shielded segment immediately pays the full serial-verification cost of §5.3. (Only 2 of 6 planned configurations completed; see §4.)

6. Bugs found in the shielded-outputs stack

Bringing shielded transactions through the full processing and byte-serialization paths — not merely building them as in-memory objects, which is all the upstream tests do — surfaced **three high-severity defects** in the `feat/shielded-outputs` branch. Each was fixed on our working branch with a runnable regression test, and written up upstream-ready (symptom → root cause → why existing tests miss it → fix → reproduction).

#	Defect	Component	Root cause	Status
1	**Shielded-output txs fail the Pedersen balance check**	`dag_builder` (`vertex_exporter.py`)	The builder gives each shielded output an independent random value-blinding and never reconciles them, so `Σ r_out ≠ 0` and `verify_shielded_balance` can never hold. Fix: reconcile the last output's blinding to the balancing residual.	Fixed
2	**Shielded txs can't be deserialized from bytes**	`hathorlib.serialization` (`generic_adapter.py`)	`GenericDeserializerAdapter` forwards every read method except `replace_remaining`, so parsing any shielded / unshield / mint / melt header throws. Fix: forward the missing method.	Fixed
3	**Full-shielded txs with > 1 input fail surjection**	`dag_builder` (`vertex_exporter.py`)	The builder constructs the surjection proof over a hard-coded **single-input** domain, but the verifier derives the domain from **all** inputs, so any fully-shielded tx with > 1 input is rejected. Fix: build the surjection domain from the real input set.	Fixed

The common thread is that all three live in code paths the upstream tests never exercise: shielded transactions are only ever inspected as in-memory objects, never driven through `verify_shielded_balance` (#1, #3) nor round-tripped through the byte-parse path used by storage and p2p (#2). Each write-up ends with the specific missing test that would have caught it. Without these fixes, none of §5.3–§5.7 would run: bug #1 blocks any shielded-output transaction, #2 blocks storing or relaying one, and #3 blocks every multi-input fully-shielded transaction — i.e. the entire high-input half of the surjection grid.

7. Conclusions & further work

The optimizations are real, and they are a consensus/storage story. With every section on, the node runs **≈ 3× faster** on the transparent baseline (up to ~4.3× on a quieter machine), and the win is dominated by **S5 (consensus + storage)** and the removal of the **redundant second full validation** in S6 — not by the Rust verification rewrite per se, which (having cut verification to ~2 % of the budget) mostly enabled the consensus layer to become the thing worth optimizing. This independently reproduces the Phase-1 bottleneck ranking.

Shielded transactions are a separate, serial regime. They run **≈ 9× slower** at 1i2o and up to **38× slower** at 8i8o, with **≈ 90 % of the cost in range-proof verification**, and the expensive axis is **outputs, not inputs** — the mirror image of transparent. Crucially, **the PR's parallelization does not reach shielded verification**: it runs serially, one proof at a time. The single clearest future lever for shielded throughput is therefore **parallelizing range-proof verification** across cores — an embarrassingly-parallel workload (N independent proofs per transaction) that neither the base implementation nor this PR exploits. A second, immediate lever sits with the application: a **narrower range-proof bit-width** (§5.8) cuts per-output verification cost and size roughly linearly, so outputs that never hold large values need not pay for a 64-bit range. That, plus a **k-tip mesh workload** to match mainnet consensus topology, and a **mainnet-sized database** to expose the real RAM/cache ceiling, are the natural Phase-4 items.

On reading these numbers. They are single-thread, single-machine, WSL-volatile figures. The ratios (≈ 3× optimized, ≈ 9–38× shielded, S5-dominant) and the per-stage attributions are the durable result; the absolute tx/s must be scaled to the reader's hardware before it means anything.